**POLYTECHNIQUE MONTRÉAL**
affiliée à l'Université de Montréal

# A lightweight canonicalization engine in C

**WILLIAM ZHANG**
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

**POLYTECHNIQUE MONTRÉAL**
affiliée à l'Université de Montréal

Ce mémoire intitulé :

**A lightweight canonicalization engine in C**

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

**Danielle DUBOIS**, présidente
**Sébastien VIGOT**, membre et directeur de recherche
**Marie COUTURE**, membre et codirectrice de recherche
**Jean TREMBLAY**, membre
**Joseph BROWN**, membre externe

## DEDICATION

*To all the past, present and future CVXPY contributors*

# ACKNOWLEDGEMENTS

Texte / Text.

# RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

# ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

# TABLE OF CONTENTS

DEDICATION . . . . . iii

ACKNOWLEDGEMENTS . . . . . iv

RÉSUMÉ . . . . . v

ABSTRACT . . . . . vi

LIST OF TABLES . . . . . ix

LIST OF FIGURES . . . . . x

LIST OF SYMBOLS AND ACRONYMS . . . . . xi

CHAPTER 1 INTRODUCTION . . . . . 1

CHAPTER 2 LITERATURE REVIEW . . . . . 4
2.1 Canonicalization . . . . . 5
2.2 Matrix stuffing . . . . . 10
2.3 Computing derivatives . . . . . 16
2.4 Coefficient extraction . . . . . 21
2.5 Sparse matrices . . . . . 22

CHAPTER 3 A lightweight canonicalization engine . . . . . 26
3.1 Fundamental data structures . . . . . 27
3.2 Atom implementations . . . . . 27
3.2.1 Generic sparse blocks . . . . . 28
3.2.2 Dense blocks . . . . . 28
3.2.3 Matrix-valued products and the Kronecker block . . . . . 29
3.3 Reusing work across passes . . . . . 29
3.4 Revisiting the limitations of matrix stuffing . . . . . 29

CHAPTER 4 Benchmark results . . . . . 31
4.1 Benchmark Setup . . . . . 31
4.2 Single-solve compilation against `cvxcore` . . . . . 32
4.3 Parametric compilation against `cvxcore` . . . . . 33
4.4 Comparison against CasADi . . . . . 33

CHAPTER 5 CONCLUSION . . . . . 37
5.1 Summary of Works . . . . . 37
5.2 Limitations . . . . . 37
5.3 Future Research . . . . . 37

REFERENCES . . . . . 38

# LIST OF TABLES


Table 2.1 A subset of the linear operators, classified as views or copies, applied to an argument $f_1$ with augmented matrix $[A_1\ b_1]$. . . . . 12

Table 2.2 Differentiation methods classified by the four choices of this section: the *expression type* (scalar or vectorized), the *mode* (forward, reverse, or both), the *assembly* of the Jacobian (formed *directly* by composing local Jacobians, or matrix-free through an *oracle* that returns only matrix–vector products and is reconstructed by seeded sweeps), and the *data format* (sparse or dense). The vectorized, forward, direct, sparse methods (lower block) need no graph coloring, unlike the matrix-free methods above them. . . . . 20

Table 2.3 Comparison of problem-data extraction methods across modeling languages. . . . . 21

Table 3.1 The representations an affine atom's Jacobian block takes. The diagonal $\mathbf{diag}(a)$ of a scaling and the Kronecker product $I_p \otimes F$ of a matrix-valued product are applied to the child Jacobian without being stored. . . . . 28

Table 4.1 Cold compilation time across all 25 benchmarks, C engine versus `cvxcore`. 32

Table 4.2 Parametric compilation, C engine versus `cvxcore`: first solve and warm re-solve. Ratio is on the re-solve. . . . . 33

Table 4.3 Single problem-data extraction across all 25 benchmarks, C engine versus CasADi (s). . . . . 35

Table 4.4 Parametric extraction, C engine versus CasADi: first extraction and warm re-solve (s). Ratio is on the re-solve. . . . . 36

# LIST OF FIGURES


Figure 1.1 Example of an affine expression tree for $Ax - b$. Variables are drawn as circles and constants (or parameters) as boxes. . . . . 2

Figure 2.1 *Before canonicalization.* CVXPY represents the DCP problem (2.4) as these expression trees. Variables are labeled with a circle, and parameters with a square. . . . . 8

Figure 2.2 *After canonicalization.* CVXPY represents the conic form (2.5) as these expression trees: the nonlinear norm atoms have become epigraph variables, leaving the objective and the three separate cone constraints affine. The variable $x$ appears as a leaf of several trees; all occurrences refer to the same variable, which owns a single block of columns in the problem data. . . . . 9

Figure 2.3 The vec convention on a matrix variable $X$ and the product $FX$, with $X, F \in \mathbf{R}^{2\times 2}$. The entries of $\mathbf{vec}(X)$, in column-major order, index the columns of the Jacobians. . . . . 11

Figure 2.4 Matrix stuffing of problem (2.4) by a recursive walk of the conic-form expression trees of 2.2: each node's matrix is obtained by applying a linear operation to the matrices of its children. . . . . 13

Figure 2.5 What each layout stores during the walk, at the node $-(Fx - g)$ of problem (2.4): a dictionary of narrow per-variable blocks, or a single matrix spanning all columns. . . . . 14

Figure 2.6 The DPP path on the objective data of problem (2.4): the parameter map $Q$ is built and cached once, and each re-solve recovers $(c, d)$ with the single sparse product $Q\tilde{\theta}$. . . . . 14

Figure 2.7 The `ignore_dpp` path on the objective of problem (2.4): the parameter leaf $\mu$ is replaced by a constant holding its current value 2.5, and the parameter-free tree is re-walked on every change of the parameters. . 15

Figure 3.1 The two passes on the $Fx - g$ subtree of the constraint tree of 2.4. The constant $F$ enters through the matmul atom, whose block $-F$ has a sparsity pattern fixed once; across solves only the parameter $g$ changes. 30

# LIST OF SYMBOLS AND ACRONYMS

| | |
|---|---|
| AD | Algorithmic Differentiation |
| AML | Algebraic Modeling Language |
| ASA | Affine-Solver-Affine |
| CSC | Compressed Sparse Column |
| CSR | Compressed Sparse Row |
| DAG | Directed Acyclic Graph |
| DCP | Disciplined Convex Programming |
| DPP | Disciplined Parameterized Programming |
| LP | Linear Program |
| MOI | MathOptInterface |
| MPS | Mathematical Programming System |
| NLP | Nonlinear Program |
| QP | Quadratic Program |
| SOCP | Second-Order Cone Program |
| SDP | Semidefinite Program |

# CHAPTER 1 INTRODUCTION

Domain specific modeling languages for convex optimization such as CVXPY [1] and YALMIP [2] allow users to formulate optimization problems in a natural mathematical syntax. A convex optimization problem parametrized by $\theta \in \mathbf{R}^p$ can be written as

$$\begin{array}{ll} \text{minimize} & f_0(x;\theta) \\ \text{subject to} & f_i(x;\theta) \leq 0, \quad i = 1,\ldots,m_1 \\ & g_j(x;\theta) = 0, \quad j = 1,\ldots,m_2 \end{array} \tag{1.1}$$

where $x \in \mathbf{R}^n$ is the optimization variable, the objective $f_0$ and inequality constraints $f_1,\ldots,f_{m_1}$ are convex and the equality constraints $g_1,\ldots,g_{m_2}$ are affine [3]. The parameter $\theta$ specifies data that can change but is fixed and given when solving an instance of the problem. We call the parametrized problem (1.1) a problem *family*; for a fixed value of $\theta$, we call it a problem *instance*.

Through a process called *canonicalization*, these high-level problem descriptions are converted to a *standard conic form* used by many general-purpose convex solvers

$$\begin{array}{ll} \text{minimize} & \frac{1}{2}x^T P(\theta)x + c(\theta)^T x \\ \text{subject to} & b(\theta) - A(\theta)x \in K \end{array} \tag{1.2}$$

where the *problem data* are the symmetric positive semidefinite matrix $P(\theta) \in \mathbf{S}^n_+$, the matrix $A(\theta) \in \mathbf{R}^{m\times n}$, the vectors $b(\theta) \in \mathbf{R}^m$ and $c(\theta) \in \mathbf{R}^n$ and the non-empty closed convex cone $K \subseteq \mathbf{R}^m$. Almost all of optimization practice can be divided into two groups, *modeling* and *solving*. The modeler's role is to formulate problems based on real-world business or engineering applications. The solver's role is to find an optimal solution given the data of a problem instance. Modeling languages serve as the communication bridge between these two groups. When improvements in solvers are made, modeling languages must adapt to allow users to benefit from them. Conversely, it is through natural formulations that modelers can express their requirements and help motivate solver improvements.

While the time taken to solve an optimization problem is often dominated by the solver, the time taken to *canonicalize* (or *compile*) a problem can also be significant, especially for large problems, or when many instances of a family are solved. In the case of CVXPY, this compilation cost lies largely in *matrix stuffing*, the final step of canonicalization where symbolic expression trees are converted to the explicit numerical problem data $P$, $A$, $b$ and $c$.

**Expression trees.** Modeling languages for convex optimization are built on top of a library of functions which can be used to describe problems. These functions are known as *atoms* in CVXPY since they are the fundamental building blocks of the modeling language [4]. An expression is defined recursively as an atom evaluated at *subexpressions*, each of which is itself another expression. Such an expression is naturally stored as an *expression tree*, with atoms at its internal nodes and variables, parameters, and constants at its leaves. After canonicalization the constraint expressions are affine and the objective is quadratic.

**Matrix stuffing in CVXPY.** CVXPY currently canonicalizes a parametrized problem in one of two ways: re-canonicalizing it from scratch for each new value of the parameters, or forming a *parameter map*, an affine map from the parameters to the problem data computed once and re-applied on each solve [5]. We cover CVXPY's matrix stuffing in detail in §2.2. For fast canonicalization, `cvxpygen`, a code-generation extension to CVXPY, can generate a custom C solver for a given problem family, suitable for real-time and embedded optimization [6].

**Matrix stuffing as a derivative.** After canonicalization, $A$ is the Jacobian of the constraints and $P$ the Hessian of the objective, so matrix stuffing amounts to computing a sparse Jacobian and Hessian. Classical methods compute the matrix with several passes of automatic differentiation, using graph coloring to group columns that can be recovered in a single pass [7, 8]. Recent vectorised formulations of AD have appeared in specific domains such as statistics [9] and optimal control [10], though not yet for matrix stuffing. We review these methods of computing sparse derivatives in §2.3.

**Other modeling languages.** Other modeling languages produce this data in different ways, a task we call *coefficient extraction*. JuMP and AMPL extract the coefficient data

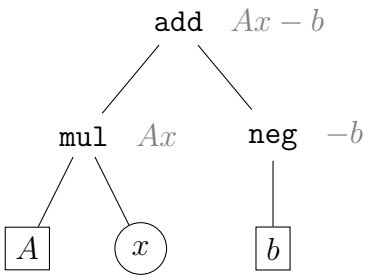



Figure 1.1 Example of an affine expression tree for $Ax - b$. Variables are drawn as circles and constants (or parameters) as boxes.

directly from the algebraic model, with AMPL also recognizing second-order cone structure and rewriting it into the solver's conic form [11–13]. Through its quadratic-programming interface, CasADi instead forms the Jacobian and Hessian by automatic differentiation, recovering their sparsity pattern by graph coloring [14]. CasADi compiles this into a reusable function, fixing the sparsity pattern once and recomputing only the nonzero values on each solve. We review these methods in much greater detail in §2.4.

**Contributions.** We make the following contributions:

1. We build `SparseDiffengine`, a C library with Python bindings that computes sparse Jacobians and Hessians of vectorized expressions, without graph coloring.
2. We implement a canonicalization backend for CVXPY that builds on `SparseDiffengine`.
3. We benchmark this backend against existing canonicalization approaches, comparing compilation time and memory use across a range of problem families.

All code is open source and available at

`https://github.com/cvxpy/cvxpy`
`https://github.com/SparseDifferentiation/SparseDiffengine`

**Thesis outline.** The rest of this thesis begins with a review of the literature on modeling languages, canonicalization, and computing derivatives in §2. In §3 we present `SparseDiffengine` and the affine and quadratic atoms it uses to canonicalize a problem into its sparse Jacobian and Hessian. In §4 we present benchmark results comparing our method to existing approaches. Finally, in §5 we summarize our contributions and discuss future improvements to the canonicalization backend.

# CHAPTER 2 LITERATURE REVIEW

**History and background** Since the early days of linear programming (LP), practitioners have needed to translate mathematical formulations into data that solvers can process. At first, this meant encoding each coefficient of the problem matrices by hand, a process that is both tedious and error-prone [15]. *Matrix generators* were introduced in the late 1960s to automate this task: they compute the coefficients through explicit loops and write them out in solver-readable formats like MPS [16, 17].

Despite matrix generators removing most of the manual labor, a gap remained. Modelers naturally think in terms of sets, indices, and algebraic expressions, while procedural code constructs the coefficient matrices entry by entry. This mismatch between how a model is expressed and how it is implemented made them difficult to verify, modify, and communicate with others [18]. *Algebraic modeling languages* such as AMPL [13], AIMMS [19] and GAMS [20] closed this gap by accepting problem descriptions that closely follow the math. For a more complete overview of the early history of modeling languages refer to [21].

More recently, modeling languages embedded in general-purpose programming languages have gained popularity, such as Pyomo [22,23] in Python, JuMP [11] in Julia, and CasADi [14] in C++. Other domain-specific modeling languages specialized for convex optimization [3] are built on *disciplined convex programming* (DCP), a set of composition rules under which the convexity of a problem can be verified automatically [4, 24]. These DCP-based languages include CVX [25] in MATLAB, CVXR [26] in R, CVXPY [1] in Python, and Convex.jl [27] in Julia.

**Structure of the chapter** The first two sections focus on CVXPY: it is the most actively maintained DCP implementation, and the work of this thesis aims to be integrated into it. §2.1 covers the *frontend* of canonicalization, which rewrites the user's functional form into a functional conic form of purely affine and quadratic expressions. §2.2 covers its *backend*, *matrix stuffing*, which assembles the problem data from this functional conic form.

Because the problem data assembled this way is exactly a sparse Jacobian of the canonicalized form, §2.3 reviews the methods available for computing sparse derivatives. §2.4 then revisits the same task from the side of other modeling systems: it presents the embedded modeling languages JuMP and CasADi as well as the algebraic language AMPL as instances of that machinery, and compares how each one extracts the problem data with the approach taken in CVXPY. Finally, §2.5 covers the sparse matrix formats used to store the problem data,

along with some useful algorithms for manipulating them.

### 2.1 Canonicalization

CVXPY is a DCP-based modeling language embedded in Python [1], which began in 2013 as an implementation of the modeling principles of CVX [25]. Three fundamental design decisions differed from CVX at the beginning. The first is modularity: a few core data structures are shared by separate components for verification, canonicalization, matrix stuffing, and solver interfaces. The second is the expression tree as the central data structure, which is formed automatically from user code through Python's operator overloading. The third is support for parameters: since canonicalization never looks at the values of constants, much of the compilation work can be reused when a problem is solved repeatedly.

These ideas matured with version 1.0, which redesigned canonicalization as a general reduction-based rewriting system [28]. The new design enabled the addition of other disciplined grammars [29–31], and the early parameter support grew into disciplined parameterized programming (DPP), a restricted grammar under which canonicalized programs can be updated efficiently [5].

The open source project has since grown through contributions from many other enthusiastic developers; a recent paper surveys the new canonicalization and modeling features introduced between versions 1.1 and 1.9 [32].

**Functional form.** A user most naturally expresses a CVXPY problem in *functional* form,

$$\begin{array}{lll} \text{minimize} & f_0(x;\theta) & \\ \text{subject to} & f_i(x;\theta) \le g_i(x;\theta), & i = 1,\dots,m_1, \\ & h_i(x;\theta) = j_i(x;\theta), & i = 1,\dots,m_2, \end{array} \tag{2.1}$$

where the optimization variables $x_1,\dots,x_k$ are collected into a single vector $x \in \mathbf{R}^n$, and likewise the parameters $\theta_1,\dots,\theta_l$ into $\theta \in \mathbf{R}^p$ [3]. As in §1, (2.1) is a problem family when $\theta$ is left symbolic and a problem instance when $\theta$ is fixed to a numerical value; it generalizes the parametric form (1.1) to the convex/concave inequalities and affine equalities CVXPY accepts. Internally, CVXPY represents each function in (2.1) as an expression tree.

**Disciplined convex programming.** When $f_0$ and all $f_i$ are convex, all $g_i$ are concave, and all $h_i$ and $j_i$ are affine, the functional form defines a convex optimization problem. To certify convexity, CVXPY uses the DCP composition rules to classify the curvature of each

function. Once all functions are classified, a problem is *DCP-compliant* if the objective and each constraint have curvatures compatible with a convex problem: a convex objective, convex $\leq$ concave inequalities, and affine equalities.

Otherwise, if a curvature is incompatible (*e.g.* $f_0$ is concave) or cannot be determined, CVXPY raises an error and terminates. Because these rules are only sufficient for convexity, a problem that is genuinely convex but not expressed in a recognized form may still be rejected.

**Composition rules.** The composition $\phi(f_1(x), f_2(x), \ldots, f_k(x))$ of an atom $\phi$ with subexpressions $f_i$ is convex when $\phi$ is convex and, for each argument $i \in \{1, \ldots, k\}$, one of the following holds:

- $\phi$ is non-decreasing in argument $i$ and $f_i$ is convex, or
- $\phi$ is non-increasing in argument $i$ and $f_i$ is concave, or
- $f_i$ is affine.

The same results hold for concave functions by swapping convex and concave. Affine expressions require $\phi$ and all of its arguments to be affine [4].

**Functional conic form.** Once a problem is verified to be DCP-compliant, the frontend of canonicalization rewrites its functional form (2.1) into an equivalent *functional conic form*

$$\begin{array}{ll} \text{minimize} & f_0(x;\theta) \\ \text{subject to} & \ell_i(x;\theta) \in \mathcal{K}_i, \quad i = 1, \ldots, m, \end{array} \tag{2.2}$$

where $f_0$ is affine or quadratic, each $\ell_i$ is affine, and each $\mathcal{K}_i$ is one of the standard convex cones: the zero, nonnegative, second-order, power, exponential, and positive semidefinite cones.

**Reductions.** This rewriting is carried out as a sequence of *reductions*. Each reduction provides an `apply` step that converts a problem into an equivalent one, and a `retrieve` step that recovers a solution of the original problem from a solution of the rewritten one [28].

A first example is `FlipObjective`, which turns a maximization problem into a minimization one by flipping the sign of the objective; its `retrieve` step simply flips the sign back.

A second example, `CvxAttr2Constr`, handles variable attributes, which are properties declared on a variable that restrict its domain. These include nonnegativity (`nonneg`), positive

semidefiniteness (`PSD`), and explicit variable bounds, amongst others. When the target solver supports an attribute natively, CVXPY leaves it for the solver to enforce; otherwise the reduction creates a new variable that carries no attributes and adds explicit constraints that enforce the attribute. Its `retrieve` step then maps the reformulated solution back onto the original attributed variable.

**Conic reformulation.** The main reduction, `Dcp2Cone`, rewrites every nonlinear atom into conic constraints. The first step of the reduction is to rewrite the problem into *Smith form*, where each nonlinear atom is replaced by a new variable coupled with an equality constraint; every atom then acts only on affine arguments [33]. For a convex atom $\phi$, the Smith form gives an auxiliary variable $t$ with the equality constraint $\phi(x) = t$. The second step relaxes this equality to the inequality $\phi(x) \leq t$. The DCP rules allow a convex atom only where its value is minimized or upper-bounded, so the solver always prefers $t$ as small as possible; the relaxation is therefore *lossless*.

The pair $(x, t)$ then lies in the *epigraph* $\{(x, t) : \phi(x) \leq t\}$ of $\phi$, so we often refer to $t$ as an *epigraph variable*. A concave atom relaxes to $\phi(x) \geq t$ instead, giving a *hypograph variable*. The final step of the reduction expresses each epigraph (hypograph) inequality as the membership of an affine function of $(x, t)$ in a convex cone [34].

**Example of reformulations.** A first example is the $\ell_2$ norm atom $\|x\|_2$, whose epigraph $\|x\|_2 \leq t$ is exactly the second-order cone constraint $(t, x) \in \mathcal{K}_{\text{soc}}$. A second example is the nondifferentiable absolute value atom $|x|$, whose epigraph $|x| \leq t$ can be written smoothly as the pair of affine inequalities $-t \leq x \leq t$. A final example is the sum of the $k$ largest entries of $x$, which expands into a small LP: the epigraph sum_largest$(x, k) \leq t$ holds if and only if there exist a scalar $q$ and a vector $u$ such that

$$q\mathbf{1} + u \geq x, \qquad u \geq 0, \qquad kq + \mathbf{1}^\top u \leq t. \tag{2.3}$$

Here $q$ acts as a threshold and each $u_i$ measures how far the entry $x_i$ rises above it, so that the sum of the $k$ largest entries is bounded by $kq + \mathbf{1}^\top u$ [3].

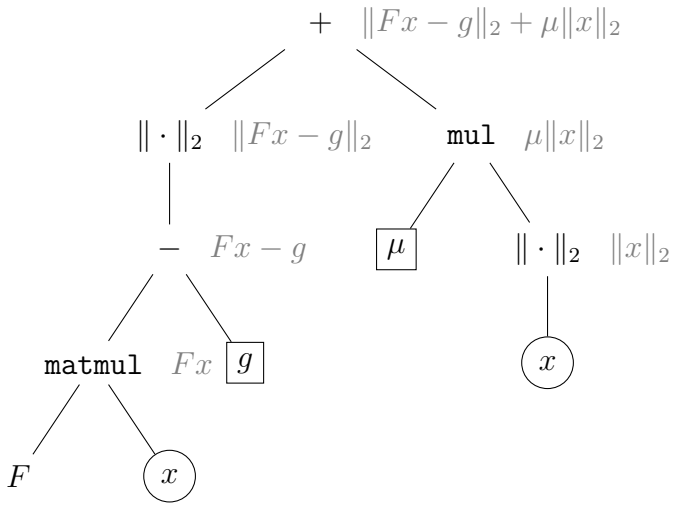
**Example of canonicalization.** To show a concrete example of the frontend of canonicalization, we consider a simple fitting with regularization problem

$$\begin{array}{ll} \text{minimize} & \|Fx - g\|_2 + \mu\|x\|_2 \\ \text{subject to} & x \succeq 0 \end{array} \tag{2.4}$$

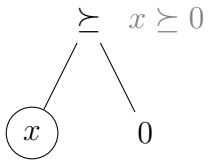
with variable $x \in \mathbf{R}^n$, fixed problem data $F \in \mathbf{R}^{m \times n}$, and parameters $g \in \mathbf{R}^m$ and $\mu \geq 0$. The problem formulation in CVXPY code is

```
import cvxpy as cp
import numpy as np
F = np.random.randn(m, n)
x = cp.Variable(n)
g = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)
objective = cp.Minimize(cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

where we also use the numerical computation library `NumPy` [35] for the data arrays. The expression tree representation of the problem is shown in 2.1.




(a) Expression tree of the objective



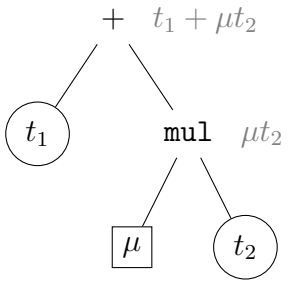

(b) Expression tree of the constraints

Figure 2.1 *Before canonicalization.* CVXPY represents the DCP problem (2.4) as these expression trees. Variables are labeled with a circle, and parameters with a square.

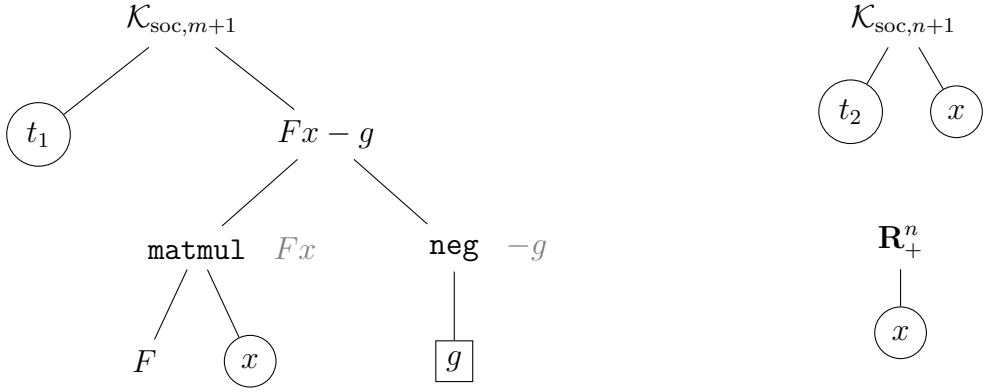
**Conic form expression tree.** After applying the reductions, the problem has been *canonicalized* to

$$\begin{array}{ll} \text{minimize} & t_1 + \mu t_2 \\ \text{subject to} & x \in \mathbf{R}^n_+ \\ & (t_1, Fx - g) \in \mathcal{K}_{\text{soc},m+1} \\ & (t_2, x) \in \mathcal{K}_{\text{soc},n+1} \end{array} \tag{2.5}$$

where $t_1$ and $t_2$ are new epigraph variables, and the cone $\mathcal{K}$ from 1.2 is the product $\mathbf{R}^n_+ \times \mathcal{K}_{\text{soc},m+1} \times \mathcal{K}_{\text{soc},n+1}$. Problem (2.5) is now in the functional conic form introduced above: every constraint is a cone membership, and the objective and constraint expressions are affine. Comparing the DCP trees of 2.1 with the conic-form trees of 2.2 shows the effect of canonicalization: the nonlinear norm atoms have been replaced by epigraph variables, leaving affine expression trees in the variables. The three rows of (2.5) remain three separate cone constraints, each carrying the affine expressions that must lie in its cone; 2.2 accordingly shows one expression tree per constraint, rather than a single combined tree.




(a) Expression tree of the objective




(b) One expression tree per constraint

Figure 2.2 *After canonicalization.* CVXPY represents the conic form (2.5) as these expression trees: the nonlinear norm atoms have become epigraph variables, leaving the objective and the three separate cone constraints affine. The variable $x$ appears as a leaf of several trees; all occurrences refer to the same variable, which owns a single block of columns in the problem data.

**References.** Michael Grant's papers and PhD thesis on CVX and DCP are the standard references on verifying convexity and canonicalization [4,24,34]. The paper by Chu et al. [36] gives another perspective of canonicalization through the *relaxed smith form*, it was used ex-

tensively to learn the concept initially. The paper by Agrawal et al. [28] on rewriting systems is a great reference for understanding reductions and the core concepts behind CVXPY. Finally, the Mosek modeling cookbook is a complementary reference for epigraph reformulations and general modeling best practices [37].

## 2.2 Matrix stuffing

CVXPY takes the perspective of matrix calculus [38] for computing the coefficient matrices of linear expressions; this is where most of the computational burden of canonicalization lies. Matrix stuffing is named for what it does: the coefficients of every canonicalized expression are stuffed into their designated slots in the sparse standard-form data handed to the solver. The name suggests bookkeeping, but the coefficients it assembles are the Jacobians and Hessians of the canonicalized expressions: matrix stuffing is a sparse derivative computation, a view developed in §2.3.

The section gives a brief summary of matrix calculus and a few examples, presents the conventions of CVXPY's backend, and closes with implementation details that become important when contrasting it with the engine of §3.

**Matrix calculus.** Matrix calculus extends differentiation to functions whose arguments and values are vectors and matrices. Systematic rules for such derivatives go back to Dwyer and Macphail [39], and there are many ways to define the derivative of a matrix-valued expression with respect to a matrix variable [38]. The simplest is to vectorize: every expression is identified with

$$\mathbf{vec}(X) = \begin{bmatrix} x_{11} & \cdots & x_{m1} & x_{12} & \cdots & x_{mn} \end{bmatrix}^T, \tag{2.6}$$

the vector that stacks the columns of $X \in \mathbf{R}^{m\times n}$, and the derivative of one vectorized expression with respect to another is an ordinary Jacobian matrix. Under this convention matrix operations become linear operators on the vectorized arguments, and their Jacobians carry structure: for example

$$\mathbf{vec}(FX) = (I \otimes F)\ \mathbf{vec}(X), \tag{2.7}$$

so the matrix product $FX$ corresponds to the Kronecker-structured, block-diagonal Jacobian $I \otimes F$. The convention is illustrated in 2.3: as an expression, a matrix variable is the identity map of its own entries, so its Jacobian is an identity block whose columns are indexed by the entries of $\mathbf{vec}(X)$, and the Jacobian of the product $FX$ is block-diagonal with one copy of $F$ per column of $X$.

`X = cp.Variable((2, 2))` $\quad\Longrightarrow\quad$ $\frac{\partial\,\mathbf{vec}(X)}{\partial\,\mathbf{vec}(X)} = \begin{array}{c} \\ x_{11} \\ x_{21} \\ x_{12} \\ x_{22} \end{array} \begin{array}{c} \begin{array}{cccc} x_{11} & x_{21} & x_{12} & x_{22} \end{array} \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{array}$

(a) A matrix variable is the identity map of its own entries.

`Y = F @ X` $\quad\Longrightarrow\quad$ $\frac{\partial\,\mathbf{vec}(Y)}{\partial\,\mathbf{vec}(X)} = I \otimes F = \begin{array}{c} \\ y_{11} \\ y_{21} \\ y_{12} \\ y_{22} \end{array} \begin{array}{c} \begin{array}{cccc} x_{11} & x_{21} & x_{12} & x_{22} \end{array} \\ \begin{bmatrix} f_{11} & f_{12} & 0 & 0 \\ f_{21} & f_{22} & 0 & 0 \\ 0 & 0 & f_{11} & f_{12} \\ 0 & 0 & f_{21} & f_{22} \end{bmatrix} \end{array}$

(b) The Jacobian (2.7) of the product $Y = FX$ is block-diagonal, one copy of $F$ per column of $X$.

Figure 2.3 The vec convention on a matrix variable $X$ and the product $FX$, with $X, F \in \mathbf{R}^{2\times 2}$. The entries of $\mathbf{vec}(X)$, in column-major order, index the columns of the Jacobians.

**Composing linear operators.** After canonicalization every expression is affine, so each node of a conic-form tree can be written in the vectorized form

$$f(x) = A\,x + b = \begin{bmatrix} A & b \end{bmatrix} \begin{bmatrix} x \\ 1 \end{bmatrix}, \tag{2.8}$$

where $x$ is the stacked vector of all variables, $A$ is the Jacobian of the node, constant since $f$ is affine, and $b$ is a constant offset. Matrix stuffing computes the augmented matrix $[A\ b]$ of every node in one bottom-up pass. A variable leaf contributes an identity block in its own column slot, a constant leaf contributes its value to the offset column, and an interior atom $g$ with arguments $f_1, \ldots, f_k$ is a linear map $\sum_i L_i f_i$ whose constant Jacobians $L_i$ apply directly to the augmented matrices of its children,

$$\begin{bmatrix} A & b \end{bmatrix} = \sum_{i=1}^{k} L_i \begin{bmatrix} A_i & b_i \end{bmatrix}. \tag{2.9}$$

Computing the problem data thus amounts to composing many linear operators over the tree: the node matrices of the figures in this section are exactly these augmented blocks, and

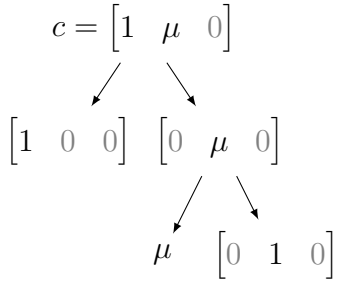
at the root of the constraint tree the augmented matrix is the problem data $[A\ b]$ itself.

**Views and copies.** The simple operators act as *views*, modifying the augmented matrix of their argument in place: negation flips the sign of every stored entry, indexing keeps the selected rows, and reshaping costs nothing at all, since it leaves the vectorization of the expression unchanged. The operators that create *copies* are mostly the bivariate atoms that carry data: one operand of a matrix product, an elementwise multiplication or a division is constant under the DCP rules, and the atom forms a matrix from that operand, multiplies it with the augmented matrix of the other, and stores the product as new data. An expository subset is classified in 2.1.

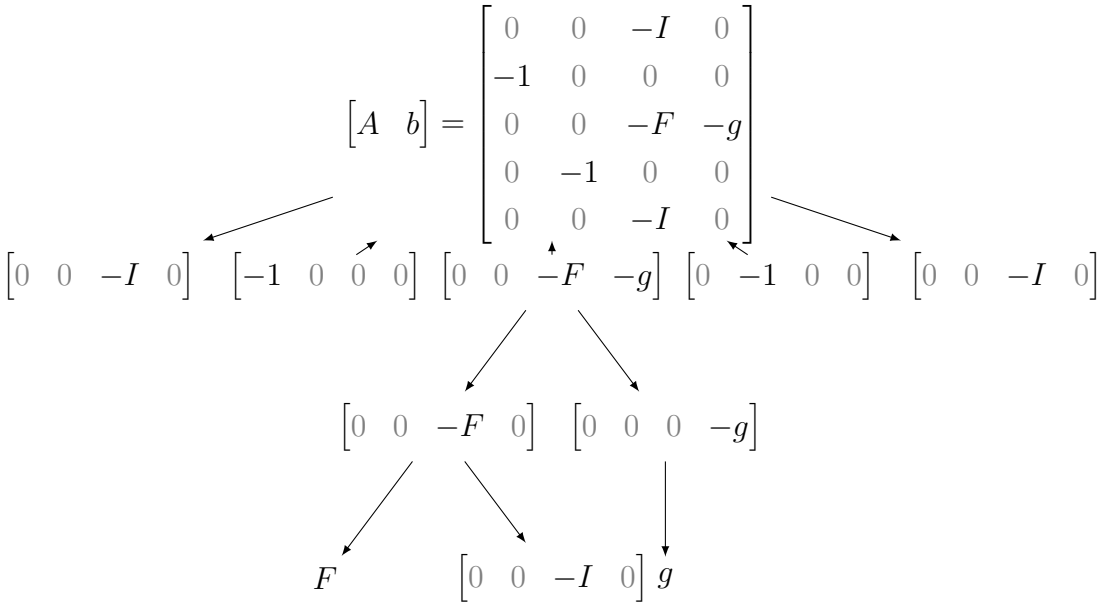
Table 2.1 A subset of the linear operators, classified as views or copies, applied to an argument $f_1$ with augmented matrix $[A_1\ b_1]$.

| **Atom** | **Operator** $L$ | **Action on** $[A_1\ b_1]$ |
|---|---|---|
| *Views* | | |
| $-f_1$ | $-I$ | flip the sign of every stored entry |
| $f_1^T$ | $P$, the vec-permutation | permute the rows |
| $f_1[\mathcal{I}]$ | $P_{\mathcal{I}}$, a row selection | keep the rows indexed by $\mathcal{I}$ |
| reshape$(f_1)$ | $I$ | none, the vectorization is unchanged |
| *Copies* | | |
| $Ff_1$ | $I \otimes F$ | multiply by a block diagonal formed from $F$ |
| $f_1B$ | $B^T \otimes I$ | multiply by a Kronecker matrix formed from $B$ |
| $c \odot f_1$ | $\mathbf{diag}(\mathbf{vec}(c))$ | multiply by a diagonal formed from $c$ |
| $f_1/c$ | $\mathbf{diag}(\mathbf{vec}(c))^{-1}$ | multiply by the reciprocal diagonal |

**Example.** The walk on the conic-form trees of problem (2.4) is traced in 2.4: the leaves contribute their blocks and offsets, the interior nodes apply their operators, and the root stacks its children at their constraint row offsets into $[A\ b]$. The difficulty that remains is doing this efficiently for arbitrary problems.




(a) Matrix stuffing of the objective




(b) Matrix stuffing tree of the constraints

Figure 2.4 Matrix stuffing of problem (2.4) by a recursive walk of the conic-form expression trees of 2.2: each node's matrix is obtained by applying a linear operation to the matrices of its children.

**Local versus global derivatives.** The augmented matrices of (2.8) can be laid out in two ways (2.5). CVXPY differentiates *locally*, one variable at a time: each node stores one coefficient block per variable appearing beneath it, keyed by the variable's identifier, and every operator is applied to each block in turn. The constant data is stored under the key $-1$, a Python idiom for the last column, the offset $b$. The columns of a block index only the entries of its own variable; only after the walk completes are the blocks shifted to their variables' column offsets and concatenated into the full matrix $[A\ b]$.

The alternative is a *global* derivative: a single Jacobian per node whose columns span the whole stacked vector $x$ from the outset. The local layout keeps the blocks small and skips

variables absent from a subtree, but every operator runs once per stored block; the engine of §3 takes the global route.

$$\left\{ \begin{array}{cc} x & 1 \\ x \mapsto \begin{bmatrix} -F \end{bmatrix}, & -1 \mapsto \begin{bmatrix} -g \end{bmatrix} \end{array} \right\} \qquad \begin{array}{cccc} t_1 & t_2 & x & 1 \\ \begin{bmatrix} 0 & 0 & -F & -g \end{bmatrix} \end{array}$$

(a) Local layout. (b) Global layout.

Figure 2.5 What each layout stores during the walk, at the node $-(Fx-g)$ of problem (2.4): a dictionary of narrow per-variable blocks, or a single matrix spanning all columns.

**Parameters and tensors.** DPP restricts parameters to enter expressions affinely, so the problem data is affine in the parameters [5], and CVXPY accounts for this with a third tensor axis: every coefficient block stores one slice per parameter entry plus one slice for the constant part, and the operators of the walk apply to each slice. Flattening the slices of the root yields the cached *parameter map*

$$\begin{bmatrix} c \\ d \end{bmatrix} = Q\,\tilde{\theta}, \qquad \tilde{\theta} = \begin{bmatrix} \theta \\ 1 \end{bmatrix}, \tag{2.10}$$

shown in 2.6 for the objective of problem (2.4); the constraint data $[A\ b]$ is recovered the same way from its own map. After a parameter change, a re-solve costs a single sparse matrix–vector product $Q\tilde{\theta}$.

$$Q = \begin{array}{c} \\ t_1 \\ t_2 \\ x \\ d \end{array} \begin{array}{c} \begin{array}{ccc} g^T & \mu & 1 \end{array} \\ \begin{bmatrix} 0 \cdots 0 & 0 & 1 \\ 0 \cdots 0 & 1 & 0 \\ 0 \cdots 0 & 0 & 0 \\ 0 \cdots 0 & 0 & 0 \end{bmatrix} \end{array} \qquad \begin{bmatrix} c \\ d \end{bmatrix} = Q\,\tilde{\theta} = \begin{bmatrix} 1 \\ \mu \\ 0 \\ 0 \end{bmatrix}$$

Figure 2.6 The DPP path on the objective data of problem (2.4): the parameter map $Q$ is built and cached once, and each re-solve recovers $(c, d)$ with the single sparse product $Q\tilde{\theta}$.

**The `ignore_dpp` path.** Problems that fall outside the DPP grammar, or that opt out of it by solving with `ignore_dpp=True`, take a simpler path. The reduction `EvalParams` replaces each parameter leaf by a constant holding its current value, and the parameter-free tree that results is stuffed as if the problem had no parameters (2.7). The path is fully general, since

arbitrary functions of parameters reduce to constants, but every change of $\theta$ repeats the whole symbolic walk even though the sparsity pattern of the problem data never changes. This is the regime the engine of §3 improves.

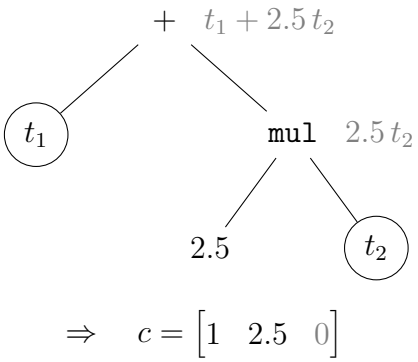



Figure 2.7 The `ignore_dpp` path on the objective of problem (2.4): the parameter leaf $\mu$ is replaced by a constant holding its current value 2.5, and the parameter-free tree is re-walked on every change of the parameters.

**Code generation.** A third option is to compile the problem ahead of time. CVXPYgen emits problem-specific C code that embeds the cached parameter map (2.10) of a DPP problem [6], continuing an earlier line of work on code generation for embedded second-order cone programs [36]. The benchmarks of §4 do not compare against it: code generation is an offline step that fixes the problem structure at compile time, a different deployment model from a runtime backend that must accept arbitrary problems.

**Implementation details.** Three implementation details, shared by CVXPY's two backends, the C++ `cvxcore` and the `SciPy` backend, stand out as limitations of the current approach; the engine of §3 does each differently.

- *External sparse libraries.* Both backends delegate every matrix operation to a general-purpose sparse library, Eigen [40] for `cvxcore` and `scipy.sparse` [41] for the `SciPy` backend. The storage formats and kernels are fixed by the library, leaving no room to exploit structure known at canonicalization time.
- *Everything is sparse.* All coefficient data is treated as sparse: dense atom data is converted to a sparse matrix on entry, and no operator carries a dense or block special case, so dense blocks pay sparse-format overhead and no BLAS kernel is ever used.
- *Quadratic objectives are separate.* Each quadratic form is swapped for a dummy variable, and the surrounding affine expression is stuffed by the same walk. The matrices

of the quadratic forms are then scaled by the resulting coefficients and reassembled into the global objective matrix $P$.

**References.** The original CVXPY paper [1] introduces the matrix calculus perspective, and the rewriting-system paper [28] situates matrix stuffing as the final step of the reduction chain. The DPP paper [5] develops the parameter map and its caching; the code generation lineage runs from the embedded second-order cone solvers of Chu et al. [36] to CVXPYgen [6].

## 2.3 Computing derivatives

Finding the rate of change, or *derivative*, of a function is a fundamental problem in many fields of science and engineering. Although computing derivatives may seem straightforward at first, there are many approaches, each with its own trade-offs [42]. These approaches differ in four basic choices: whether the function is treated as a *scalar* or *vectorized* expression, whether derivatives are propagated *forward* or in *reverse*, whether the Jacobian or Hessian is assembled *directly* or accessed through an *oracle* of matrix–vector products, and whether the resulting matrix is *sparse* or *dense*.

The problem data for an optimization model is often large and sparse, so we place particular emphasis on computing sparse derivatives in this section. The standard approach in the literature is scalar and oracle-based differentiation [43], where graph coloring is used to recover a sparse matrix in far fewer oracle evaluations. This thesis instead proposes to compute these sparse derivatives directly on a vectorized graph whose sparsity pattern follows from the structure of the operations, so no coloring is needed.

The remainder of the section is structured as follows: we begin with a brief survey of the standard numerical and symbolic methods, before turning to algorithmic differentiation. Finally, we gather a range of relevant works, organizing them in a table according to their particular approach to differentiation.

**Numerical differentiation.** Other than computing derivatives by hand, the simplest method is to use *numerical differentiation*, which approximates the derivative of a function at a point by evaluating the function at nearby points using a *finite difference* formula similar to either

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \tag{2.11}$$

or

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}, \tag{2.12}$$

where $h$ is a small step size. Although this approach is straightforward to implement and can achieve reasonable accuracy, there are important downsides to consider. First, it requires a linear number of function evaluations with respect to the number of input variables, which is expensive for high-dimensional functions. Second, it can easily lead to *truncation errors* (when $h$ is too large) or *rounding errors* (when $h$ is too small) in the computed derivative if the step size is not carefully chosen [44]. Despite these downsides, numerical differentiation is still widely used in practice, especially for verifying the correctness of derivative computations (also known as a *derivative checker*) [45].

**Symbolic differentiation.** Mathematical expressions can be manipulated automatically using simple derivative rules of calculus, such as the product and sum rule. This *symbolic* approach to differentiation is used in computer algebra systems such as Mathematica [46] and Maple [47] as well as the SymPy library in Python [48]. Unlike its numerical counterpart, symbolic differentiation tools compute derivatives as closed-form expressions, which can be simplified and evaluated at any arbitrary point. These closed-form expressions are particularly useful for deriving analytic solutions of optimization problems, and can be further used in mathematical proofs and analyses.

However, a naive implementation of symbolic differentiation can quickly form exponentially large symbolic expressions, which is impractical and inefficient for many problems [43]. Interestingly, when intermediate expressions are stored as *common subexpressions*, and the expression is represented with a *directed acyclic graph* (DAG) instead of a tree, symbolic differentiation performs the same set of operations as *reverse mode differentiation* [49].

**Algorithmic differentiation.** *Algorithmic differentiation* (AD) treats the function as a *computational graph* of elementary operations, each with a known local derivative rule. The chain rule is then applied to combine these local derivatives throughout the graph, giving the derivative of the whole function. AD is also commonly referred to as *automatic differentiation*; we prefer the term *algorithmic* to emphasize that AD differentiates the algorithm that computes a function, rather than a numerical approximation or a symbolic formula [43]. In this broad sense, AD can refer to any method that applies the chain rule over a computational graph. In the remainder of this section, we introduce the standard scalar and oracle-based differentiation methods, then contrast them with vectorized and direct approaches.

**Scalar differentiation.** The earliest and most foundational form of AD operates on a *scalar* computational graph, where every node is a single elementary operation (an addition, a multiplication, or a univariate function such as sin or exp), and every edge carries the

corresponding scalar partial derivative [43, 50]. Since each elementary operation has a simple and known derivative, the chain rule reduces to multiplying and summing scalars along the graph. Early AD tools such as ADOL-C [51] and CppAD [52] form the scalar graph by recording a runtime trace of every elementary operation. Such a trace can be enormous: a single matrix–vector product $Ax$ unrolls into one scalar multiplication for every entry of $A$, together with the additions that sum them, so it quickly reaches millions of nodes for a realistic problem.

**Modes of differentiation.** The chain rule can be applied in different orders, all giving the same result. The two best-known are *forward mode* (or *tangent* mode), which propagates derivatives from the inputs toward the outputs, and *reverse mode* (or *adjoint* mode), which propagates them from the outputs back toward the inputs [42]. The two modes have asymmetric costs: in forward mode the work scales with the number of inputs, and in reverse mode with the number of outputs. Reverse mode is therefore preferred when there are far fewer outputs than inputs, such as computing the gradient of a scalar function. Other mixed-ordering modes exist, commonly grouped under the umbrella term *cross-country* elimination [43]. However, finding the best ordering on a general graph is difficult, so these modes are rarely used in practice [53].

**First- and second-order derivatives.** For a vector-valued map $F : \mathbf{R}^n \to \mathbf{R}^m$ of an input $x \in \mathbf{R}^n$, the first-order derivative is the *Jacobian* $J \in \mathbf{R}^{m\times n}$ of partial derivatives of the outputs with respect to the inputs,

$$J_{ij} = \frac{\partial F_i}{\partial x_j}, \qquad i = 1, \dots, m, \quad j = 1, \dots, n. \tag{2.13}$$

For a scalar function $f : \mathbf{R}^n \to \mathbf{R}$, the second-order derivative is the *Hessian* $H \in \mathbf{R}^{n\times n}$ of second partial derivatives,

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}, \qquad i, j = 1, \dots, n. \tag{2.14}$$

The Hessian is the Jacobian of the gradient of $f$, and is symmetric whenever $f$ is twice continuously differentiable. It is therefore obtained by differentiating the gradient, with no method beyond those used for the Jacobian.

**Oracle-based differentiation.** Instead of $J$ itself, AD returns its action on a vector: the Jacobian-vector product (JVP) $Jv$ in forward mode, and the vector-Jacobian product

(VJP) $J^\top w$ in reverse mode (for the Hessian, the analogous Hessian-vector product $Hv$ comes from one forward pass over one reverse pass) [42]. These matrix-vector products are obtained without ever forming $J$, each at about the cost of one function evaluation. Borrowing terminology from numerical linear algebra, we call this the *matrix-free*, or *oracle*, view of AD. The oracle is sufficient for many applications; they never form the matrix and use only its action on vectors, as in backpropagation for training neural networks or Krylov methods for linear systems.

**Recovering the Jacobian.** Recovering $J$ explicitly requires *probing* the oracle with a sequence of seed vectors: the $n$ unit seeds recover it column by column in forward mode, and the $m$ unit seeds recover it row by row in reverse mode (the Hessian is recovered the same way, though its symmetry permits fewer seeds) [43]. When the Jacobian has many rows or columns this becomes prohibitively expensive, and reducing the number of probes is exactly the purpose of the graph-coloring methods described next. Crucially, coloring operates on the oracle alone, independently of how the underlying graph is built, so even fully vectorized frameworks rely on it when they need an assembled sparse Jacobian.

**Graph coloring.** The cost of probing drops when several seeds share a single pass. Two columns of $J$ with no nonzero in a common row are *structurally orthogonal*, so one seed recovers both without their nonzeros colliding. Finding the fewest such groups is a graph-coloring problem on the column intersection graph, whose edges join columns that share a row [7, 8, 54]. The Hessian compresses further: its symmetry admits dedicated star and acyclic colorings that use fewer colors than the general Jacobian case [55].

**Sparse derivatives.** An oracle-based sparse Jacobian is obtained in three stages: *detect* the sparsity pattern, *compress* it by coloring, and *propagate* the compressed seeds through the oracle to recover the nonzero entries [56]. The pattern is computed once and reused, so its cost is amortized across repeated evaluations of the same function. This pipeline underlies sparse AD in oracle-based tools such as CasADi [14], which we revisit in §2.4.

**Vectorized differentiation.** More recent frameworks coarsen the graph so that each node is a whole vector or matrix operation rather than a single scalar. The local derivative of such a node is then a matrix, a block of the Jacobian, and these blocks are combined by the rules of matrix calculus [38]. The resulting graph has far fewer nodes than the scalar trace, which lets frameworks such as JAX [57] differentiate large programs efficiently. Vectorization alone,

however, does not change the assembly: these frameworks still expose only the oracle, and still rely on coloring when they need an assembled sparse Jacobian.

**Direct differentiation.** Direct methods go a step further and abandon the oracle altogether. Rather than returning products, they propagate the local Jacobian blocks themselves through the graph and combine them to form $J$ directly, so neither probing nor coloring is needed. Vectorization is the enabler: it gives each block algebraic structure, whereas a scalar node carries only a 0/1 dependency. Recent works apply this idea in statistics [9] and optimal control [10]. Direct propagation wins when the local blocks are well structured and intermediate fill-in stays bounded, while coloring through the oracle can still be cheaper when the intermediates densify. Which assembly is preferable ultimately follows the solver that consumes the result: the conic solvers targeted in this thesis require the matrix explicitly as data, which is exactly what the direct approach produces.

Table 2.2 Differentiation methods classified by the four choices of this section: the *expression type* (scalar or vectorized), the *mode* (forward, reverse, or both), the *assembly* of the Jacobian (formed *directly* by composing local Jacobians, or matrix-free through an *oracle* that returns only matrix–vector products and is reconstructed by seeded sweeps), and the *data format* (sparse or dense). The vectorized, forward, direct, sparse methods (lower block) need no graph coloring, unlike the matrix-free methods above them.

| **Method / work** | **Expression type** | **Mode** | **Assembly** | **Data format** |
|---|---|---|---|---|
| ADOL-C, CppAD [51, 52] | scalar | both | oracle | dense |
| Tapenade, Enzyme [58, 59] | scalar | both | oracle | dense |
| CasADi [14] | scalar / vectorized | both | oracle | sparse |
| JAX [57] | vectorized | both | oracle | dense |
| PyTorch [60] | vectorized | reverse | oracle | dense |
| `torch-sla` [61] | vectorized | reverse | oracle | sparse |
| CPR, Coleman–Moré [7, 54] | scalar | forward | oracle | sparse |
| Hill–Dalle [56] | scalar | both | oracle | sparse |
| Vector forward mode [62] | scalar | forward | oracle | dense |
| Zou and Jiang [10] | vectorized | forward | direct | sparse |
| Kwok et al. [9] | vectorized | forward | direct | sparse |
| ∇SD [63] | vectorized | forward | direct | sparse |
| **This thesis** | **vectorized** | **forward** | **direct** | **sparse** |

**Overview of recent works.**

**References.** The book by Griewank and Walther [43] is a comprehensive reference on the theory and practice of scalar automatic differentiation, including the graph-elimination view

of Jacobian accumulation. The book by Naumann [50] is another great reference that focuses on the implementation of AD in software, with a particular emphasis on the use of operator overloading and source code transformation techniques. The survey by Baydin et al. [42] is an excellent and relatively up-to-date overview of the various methods for automatic differentiation in machine learning. For the combinatorial side of sparse derivative computation, the survey of Gebremedhin et al. [8] is the standard entry point, and the illustrated guide of Hill and Dalle [64] gives an accessible modern tutorial on the detection–coloring–compression pipeline.

## 2.4 Coefficient extraction

**`JuMP.`**

**`AMPL.`**

Table 2.3 Comparison of problem-data extraction methods across modeling languages.

| | **JuMP** | **AMPL** | **CasADi** | **CVXPY** |
|---|---|---|---|---|
| Front-end paradigm | Algebraic modeling | Algebraic, file-based | Symbolic graphs + AD | Disciplined convex (DCP) |
| Problem classes | LP, MILP, QP, conic, NLP | LP, MILP, NLP (conic via recent drivers) | NLP, QP / conic (QP-shaped) | Convex: LP, QP, SOCP, SDP, EXP |
| Path to conic solvers | Native cones + bridges (MOI) | Solver-side, recent drivers | `soc` helper on the QP path | Automatic DCP reduction |
| Problem-data mechanism | Eager, per scalar constraint | `.nl` file; nonlinear via solver AD | Explicit $H, A$ by AD on the graph | Affine-tree stuffing + parameter map |

**`CasADi.`**

**References.** The MathOptInterface paper [65] is the best reference on the design of JuMP's solver abstraction layer and its bridge system. David Gay's technical reports [66, 67] remain the authoritative description of the `.nl` format and the AMPL solver interface. Finally, the CasADi paper [14] is a self-contained reference covering its expression graphs, AD implementation, and solver interfaces.

## 2.5 Sparse matrices

Matrices that contain mostly zero entries are known as *sparse matrices*. Storing only the nonzero entries makes memory and work scale with their number rather than with the product of the dimensions. The simplest sparse matrix data structure is the *coordinate* or *triplet* format. It consists of two integer index arrays `rows` and `cols` and a floating point value array `vals` of length equal to the number of *nonzero entries* in the matrix.

For example, the matrix

$$S = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 2 & 0 & 3 & 4 \\ 0 & 5 & 6 & 0 \end{bmatrix} \tag{2.15}$$

can be represented in coordinate format as

```
int    rows[] = {0, 1, 1,  1,  2,  2};
int    cols[] = {1, 0, 2,  3,  1,  2};
double vals[] = {1, 2, 3, 4,  5,  6};
```

where we use zero-based indexing and array declaration notation from the C programming language [68]. The coordinate format is the easiest to understand and form, but it is not effective for most operations on sparse matrices; the *compressed sparse row* or CSR format is commonly used instead.

**Compressed sparse row format.** The CSR format differs from the coordinate format in that it replaces the `rows` array with a row pointer array `idxptr` of length equal to the number of rows plus one. The row pointer array indicates the starting index of each row in the `cols` arrays. For example, the matrix in (2.15) can be represented in CSR format as

```
int  idxptr[] = {0, 1,         4,      6};
int    cols[] = {1, 0, 2,  3,  1,  2};
double vals[] = {1, 2, 3, 4,  5,  6};
```

where we deliberately format `idxptr` to align with the start of each row in the `cols` arrays. The benefits of the CSR format are more than a reduced size of the `idxptr` array compared to the `rows` array. In the CSR format, accessing a row takes constant time and is very *cache-friendly* since the nonzero entries are stored contiguously in memory. For this reason, the CSR format is the default for *iterative methods* to solve sparse linear systems [69].

**Compressed sparse column format.** However, one downside of the CSR format is that accessing a column is much more difficult; one would need to look at every row to find the

nonzero entries in a column. An alternative format is the *compressed sparse column* or CSC format, which swaps the role of the `idxptr` array to refer to column indices instead. In contrast to iterative methods, the CSC format is the preferred format for *direct methods* to solve sparse linear systems [70].

**Transpose.** The CSC arrays of a matrix read exactly as the CSR arrays of its transpose: compressing the columns of $S$ is the same as compressing the rows of $S^T$. For (2.15),

```
int  idxptr[] = {0, 1,    3,     5,  6};
int    rows[] = {1, 0, 2,  1,  2,  1};
double vals[] = {2, 1, 5, 3,  6,  4};
```

is at once the CSC representation of $S$ and the CSR representation of $S^T$. Transposition is therefore a conversion between the two formats. It requires no sorting: one pass over the nonzeros counts the entries of each column to build the new `idxptr`, and a second pass scatters the entries into place, $O(m + n + \text{nnz})$ time in total [70, 71]. The entries of each output row come out sorted by column, so transposing twice is also the standard way to restore a matrix with unordered indices to canonical form.

**Which format to use?** The access pattern of the algorithm decides: a computation that traverses rows wants CSR, one that traverses columns wants CSC. Since conversion costs a single counting pass, keeping the same matrix in both formats is common. The engine of §3 does exactly this, storing Jacobians in CSR and pairing the two formats in its matrix products.

**Matrix-vector products.** Both compressed formats multiply a dense vector in one pass over the nonzeros, but the loop must follow the storage. In CSR each output entry is a sparse dot product, gathering from $x$ row by row:

```
/* y = S @ x, with y zeroed */
for (int i = 0; i < m; i++)
{
    for (int k = idxptr[i]; k < idxptr[i + 1]; k++)
    {
        y[i] += vals[k] * x[cols[k]];
    }
}
```

In CSC the roles reverse: the loop visits column $j$ and *scatters* the products `vals[k]` $\cdot\, x_j$ into `y[rows[k]]`, accumulating the output instead of gathering the input.

**Matrix-matrix products.** The product of two sparse matrices is harder than the operations above because the sparsity pattern of the output is not known in advance. Implementations therefore split the work in two phases: a *symbolic* phase determines the pattern and allocates the output, and a *numeric* phase fills its values [71, 72]. The split rewards reuse, and the engine of §3 is built around it: the symbolic phase runs once per problem family and only the numeric phase repeats.

Gustavson's algorithm [71] computes $C = AB$ with both operands in CSR, row by row: row $i$ of $C$ is the sum of the rows of $B$ selected and scaled by the nonzeros of row $i$ of $A$, merged in a dense accumulator $w$. Take $C = SS^T$, where $S^T$ in CSR is the transpose arrays above. Row 2 of $S$ holds 5 at column 1 and 6 at column 2, so row 2 of $C$ is $5\cdot$(row 1 of $S^T$)$+6\cdot$(row 2 of $S^T$): the first term sets $w[0] = 5$ and $w[2] = 25$, the second sets $w[1] = 18$ and grows $w[2]$ to 61. The cost is one multiplication per term merged, but the output indices appear in the order they are first touched, here $[0, 2, 1]$; transposing twice sorts them when canonical form is needed.

The alternative is to compute each entry as a sparse dot product, $c_{ij} = (\text{row } i \text{ of } A) \cdot (\text{column } j \text{ of } B)$, which asks for $A$ in CSR and $B$ in CSC; for $C = SS^T$ both roles are served by the CSR arrays of $S$ alone. Entries whose row and column share no index are structural zeros and cost only the scan: $c_{01} = 0$ since rows 0 and 1 of $S$ have no common column, while rows 1 and 2 meet at column 2, giving $c_{12} = 3 \cdot 6 = 18$. This form rescans its operands for every candidate entry, where Gustavson's merge touches each term once; in exchange it needs no accumulator and produces each row sorted.

**Other formats.** The coordinate, CSR and CSC formats are the most widely used sparse matrix formats, but many others exist for different specialized use cases. The scientific Python sparse library `scipy.sparse` [41] allows users to form other types of sparse matrices such as the list of lists (LIL) matrix, the dictionary of keys (DOK) matrix, and the block-sparse row (BSR) matrix.

**References.** The standard references treat the solution of sparse linear systems, a task this thesis never performs, but the data structures and kernels they develop apply unchanged. The book by Davis [70] was used extensively in developing the engine of §3; the book by Duff, Erisman and Reid [73] covers the history and development of the field. Gustavson's

paper [71] is the origin of the row-wise product and the sort-free transposition of this section. Finally, Gill, Murray, Saunders and Wright [74] survey the applications of sparse matrix methods in optimization.

# CHAPTER 3 A lightweight canonicalization engine

This chapter presents the `SparseDiffEngine` and how it forms the constraint matrix $A$ from the canonicalized expression trees of §2.1. We reframe the matrix stuffing of §2.2 as the computation of a single sparse Jacobian, assembled by one recursive rule applied atom by atom.

After canonicalization the constraints are affine, so $A$ is their Jacobian and is constant in $x$. Matrix stuffing walks the expression tree accumulating each atom's linear coefficient into $A$; this accumulation is exactly a bottom-up computation of that constant Jacobian.

The rule behind it is that every affine atom is a linear operator $L$ with a constant offset. If a subexpression $z$ has Jacobian $J_z$, then the atom's output $Lz + c$ has Jacobian $LJ_z$, so the Jacobian of a node is its atom's operator applied to the Jacobians of its children,

$$J_{\text{node}} = L_{\text{atom}} \begin{bmatrix} J_{\text{left}} & J_{\text{right}} \end{bmatrix}. \tag{3.1}$$

Every atom in this chapter is one choice of $L$: addition sums its children, negation flips their sign, transpose permutes the rows, and the matmul $Fx$ left-multiplies by $F$. A single pass from the leaves to the root composes these operators into $A$.

Because the constraints are affine, the Jacobian is constant and its sparsity pattern is fixed by the composition of the atoms' operators. We read that pattern off directly, in one structural pass, rather than discovering it by probing the Jacobian one column group at a time as the coloring-based methods of §2.3 do.

This walk is the one CVXPY's `cvxcore` already performs, applying each atom's linear operator to assemble the problem data. The engine keeps the walk but changes it in two ways, each developed below. It splits the single walk into a structural pass that fixes the sparsity pattern of $LJ_z$ once per problem family and a value pass that refills only the numbers on each solve (§3.3). It also stores each Jacobian block in the representation that matches its structural class, applying the dense and Kronecker-structured operators without ever forming them (§3.2).

§3.1 describes the data structures and §3.2 the atom implementations. §3.3 develops the caching the two passes enable, and §3.4 returns to the limitations of §2.2. We assume the sparse-matrix background of §2.5.

### 3.1 Fundamental data structures

The main data structure in the engine is the expression struct.

```
typedef struct expr {
    int    d1, d2, size; /* shape (d1 x d2) and size = d1*d2 */
    int    n_vars;       /* total number of variables     */
    int    var_id;       /* offset into u, or NOT_A_VARIABLE */

    struct expr *left;   /* left child (NULL for leaves)  */
    struct expr *right;  /* right child (NULL for leaves) */

    double    *value;    /* forward-pass output, size entries */
    matrix    *jacobian; /* size x n_vars Jacobian        */

    forward_fn     forward;
    jacobian_init_fn jacobian_init_impl;
    eval_jacobian_fn eval_jacobian;
} expr;
```

Each node carries a `value`, the output of its forward pass, and a Jacobian. The Jacobian is held through a polymorphic `matrix` type: a base struct of shape and a small table of operator function pointers, embedded as the first member of each concrete representation. The representation an atom uses is fixed once during initialization and is the subject of §3.2. Each node also implements three functions: `forward`, `jacobian_init`, and `eval_jacobian`.

**Leaf nodes.** The leaves are variables and parameters.

### 3.2 Atom implementations

An atom takes one or two expression operands and produces an expression. Each atom implements the three functions: `forward` computes the value, `jacobian_init` fixes the Jacobian's sparsity pattern once, and `eval_jacobian` fills its values on each solve. The affine atoms number in the dozens, but their Jacobian blocks fall into the few representations of 3.1. We organize the atoms by representation and give one worked atom for each, describing the rest in prose. Two further operators, the diagonal of an elementwise scaling and the Kronecker product of a matrix-valued product, are applied to the child Jacobian without ever being stored. Throughout, `x` and `y` denote variables, `a` a parameter, and `F` a constant matrix, all of compatible shape.

| Representation | Atoms | Jacobian block |
|---|---|---|
| Generic sparse | `add`, negation, transpose, index, sum, stack | selection or sum of the child's rows |
| Permuted dense | matmul $Fx$ with dense $F$ | the dense block $F$ at $x$'s columns |
| Block-stacked dense | matmul $FX$, matrix-valued $X$ | a stack of $p$ copies of $F$ |

Table 3.1 The representations an affine atom's Jacobian block takes. The diagonal $\mathbf{diag}(a)$ of a scaling and the Kronecker product $I_p \otimes F$ of a matrix-valued product are applied to the child Jacobian without being stored.

### 3.2.1 Generic sparse blocks

The default representation is a generic sparse matrix in CSR form (§2.5). A variable leaf of size $n$ has the identity as its Jacobian, stored as an $n \times n_{\text{vars}}$ selection matrix with one nonzero per row. The atoms whose blocks relabel, select, or add the child's rows keep this representation: the output has a nonzero wherever an input does, so its pattern is a rearrangement of the children's.

Addition is the template every such atom follows. The forward pass evaluates the children and sums their values. `jacobian_init` allocates the output Jacobian, whose number of nonzeros is at most the sum of the children's, and fixes its pattern. `eval_jacobian` sums the children's Jacobians into it.

Negation negates an expression elementwise. The forward pass flips the sign of each value; the Jacobian has the child's pattern, so `jacobian_init` copies it and `eval_jacobian` negates its values. Transpose (`cp.transpose(x)`) only relabels the output entries, so its Jacobian is a fixed row permutation of the child's: `jacobian_init` records the permutation and `eval_jacobian` copies the child's values into the permuted rows. The vertical stack `cp.vstack([x, y])` needs no new code, using the identity `vstack([A, B]) = transpose(hstack([trans transpose(B)]))`. Reshape, indexing, summation, and broadcasting are of the same kind: each fixes an output pattern that selects or accumulates the child's entries and refills it by copying or adding values.

### 3.2.2 Dense blocks

When a constant dense matrix $F \in \mathbf{R}^{m \times n}$ multiplies a vector expression, $Fx$, the Jacobian is $F$ itself, placed at the columns of $x$'s variables. A dense $F$ stored as a generic sparse matrix carries the full CSR index overhead for a block with no zeros to exploit. The engine stores it instead as a *permuted dense* block: a single $m \times n$ dense array together with the global rows and columns it occupies,

$$M[\,\text{row}[i],\ \text{col}[j]\,] = F[i, j], \tag{3.2}$$

with every other entry zero. `jacobian_init` places the block once and `eval_jacobian` refills its $mn$ values, both by applying $F$ to the child's Jacobian following (3.1).

A parameter that scales an expression elementwise, $a \odot x$, is the diagonal case: its Jacobian is $\mathbf{diag}(a)$. The engine never forms $\mathbf{diag}(a)$; it applies the operator to the child Jacobian $J$ as $\mathbf{diag}(a)\,J$, scaling row $i$ of $J$ by $a_i$. Only the vector $a$ is stored.

### 3.2.3 Matrix-valued products and the Kronecker block

The same matmul atom handles a matrix-valued operand $FX$, where $X$ has $p$ columns. Vectorizing column by column,

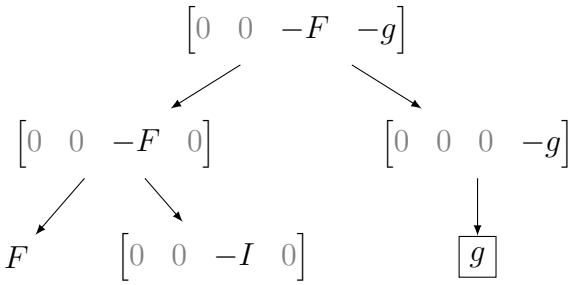
$$\operatorname{vec}(FX) = (I_p \otimes F)\,\operatorname{vec}(X), \tag{3.3}$$

so the operator is the Kronecker product $I_p \otimes F$, a block-diagonal matrix with $p$ copies of $F$ on its diagonal, and the Jacobian is $(I_p \otimes F)\,J_X$ by (3.1).

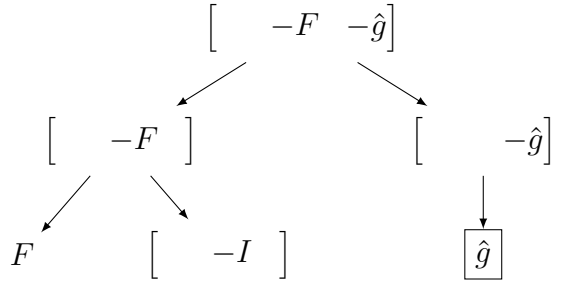
Forming $I_p \otimes F$ is wasteful: it is $p$ times the size of $F$ and completely structured. The engine never forms it. It loops over the $p$ diagonal blocks, applies the single stored $F$ to the corresponding block of $J_X$, and collects the $p$ results as a vertical stack of permuted dense blocks. The engine's own note records the effect: it forms the Kronecker product "only conceptually," which "led to a 100x speedup in the initialization of the Jacobian sparsity pattern." The stacked representation carries this structure through the rest of the walk, so an atom above a matrix-valued product also operates block by block rather than on a materialized Kronecker product.

## 3.3 Reusing work across passes

## 3.4 Revisiting the limitations of matrix stuffing




(a) Structural pass: the matmul block $-F$ inherits the pattern of $F$ and is independent of $g$.




(b) Value pass: only the values are written into that pattern, here for $g = \hat{g}$. Between solves only the parameter block $-\hat{g}$ changes; the matmul block $-F$ is untouched.

Figure 3.1 The two passes on the $Fx - g$ subtree of the constraint tree of 2.4. The constant $F$ enters through the matmul atom, whose block $-F$ has a sparsity pattern fixed once; across solves only the parameter $g$ changes.

# CHAPTER 4 Benchmark results

This chapter evaluates the C engine on the problems CVXPY users actually compile. We measure canonicalization end to end—from a built expression tree to solver-ready problem data—in the two regimes that matter in practice: the cold, single compilation of a fresh problem, and the warm re-compilation of a parametrized problem whose parameter values change between solves. The engine is compared against CVXPY's default `cvxcore` backend in both regimes (§4.2, §4.3), and, in the same two regimes, against CasADi, which obtains the same problem data by algorithmic differentiation of a symbolic graph (§4.4).

## 4.1 Benchmark Setup

We benchmark on the public CVXPY benchmark suite [75], 25 real-world problems (linear and quadratic programs, least squares, portfolio models, robust regression, and conic and semidefinite programs) maintained under *airspeed velocity* (`asv`) [76] with seeded, reproducible data. All 25 problems run on both backends; three of them exercise the `kron` atom, whose support the engine gained late in this work.

We time *canonicalization* only—the construction of the problem data $(A, b, c, P)$. Concretely we time `get_problem_data` with the target solver fixed to Clarabel [77]: the `CPP` (`cvxcore`) baseline is the default DPP path, and the C engine is selected with `ignore_dpp`.

```
prob.get_problem_data(solver=cp.CLARABEL)              # CPP baseline (DPP
    tensor)
prob.get_problem_data(solver=cp.CLARABEL, ignore_dpp=True) # our C engine
```

All measurements run single-threaded (BLAS thread pools pinned to one thread) on a MacBook Pro (15-inch, 2018): 6-core Intel Core i7-8850H at 2.6 GHz, 16 GB RAM, macOS 15.7, with Python 3.14, CVXPY 1.10 (dev), NumPy 2.4 [35], and SciPy 1.17 [41]; the CasADi comparison uses CasADi 3.7.2 [14]. Reported times are medians of three fresh compilations (twenty re-evaluations for the re-solve columns). Versions are pinned and the suite and engine are public, so the configuration is reproducible.[1]

We report three comparisons: single-solve compilation against `cvxcore` (§4.2), parametric compilation against `cvxcore` across the first solve and re-solves (§4.3), and both regimes

[1] The complete harness, per-problem CasADi models, mechanism probes, and raw result files are published at `https://github.com/SparseDifferentiation/Jacobian-accumulation` (directory `canonicalization/`).

against CasADi (§4.4). In every table the ratio is C engine/baseline; a value below 1 favors the engine, and the final row is the geometric mean of the per-problem ratios.

### 4.2 Single-solve compilation against `cvxcore`

A fresh problem is formed and canonicalized once from scratch (Table 4.1).

Table 4.1 Cold compilation time across all 25 benchmarks, C engine versus `cvxcore`.

| Benchmark | `CPP` (s) | C engine (s) | Ratio |
|---|---|---|---|
| UnconstrainedQP | 9.32 | 0.08 | **0.01** |
| Cajas | 19.66 | 1.06 | 0.05 |
| FactorCovarianceModel | 5.24 | 0.73 | 0.14 |
| SVMWithL1Regularization | 11.16 | 1.60 | 0.14 |
| Yitzhaki | 3.07 | 0.47 | 0.15 |
| HuberRegression | 10.97 | 1.61 | 0.15 |
| LeastSquares | 8.87 | 1.42 | 0.16 |
| SimpleQP | 10.49 | 1.69 | 0.16 |
| SimpleScalarParametrizedLP | 5.29 | 0.92 | 0.17 |
| SimpleLP | 30.07 | 5.29 | 0.18 |
| SDPSegfault1132 | 59.75 | 12.47 | 0.21 |
| SimpleFullyParametrizedLP | 2.12 | 0.45 | 0.21 |
| ParametrizedQP | 10.19 | 2.25 | 0.22 |
| TvInpainting | 3.57 | 1.06 | 0.30 |
| SlowPruning | 6.88 | 2.15 | 0.31 |
| SemidefiniteProgramming | 5.38 | 1.93 | 0.36 |
| ConeMatrixStuffing | 10.34 | 3.76 | 0.36 |
| QuantumHilbertMatrix | 3.89 | 1.64 | 0.42 |
| SmallMatrixStuffing | 6.52 | 2.77 | 0.42 |
| ParamSmallMatrixStuffing | 0.48 | 0.21 | 0.43 |
| ParamConeMatrixStuffing | 0.31 | 0.14 | 0.45 |
| Murray | 5.79 | 3.43 | 0.59 |
| CVaR | 34.80 | 25.31 | 0.73 |
| OptimalAdvertising | 6.06 | 5.20 | 0.86 |
| ConvexPlasticity | 2.09 | 2.13 | **1.02** |
| **Geometric mean** | | | **0.24** |

The engine is faster on 24 of the 25 problems, between 1.2× and 124×, with a geometric-mean speedup of 4.2×. The wins are largest where `cvxcore`'s per-expression tensor construction dominates (portfolio and regression models with many scalar constraints, and UnconstrainedQP's tiny problem wrapped in a large `kron` expression); they shrink toward

parity on problems whose cost is a single huge data copy (OptimalAdvertising, CVaR) or whose compilation is dominated by CVXPY's Python-side reductions rather than either backend (ConvexPlasticity, the one row at parity).

### 4.3 Parametric compilation against `cvxcore`

On the parametric subset we report the first solve and the re-solve after reassigning parameter values on a warm cached chain (Table 4.2). The re-solve is the engine's target regime; its ratio is the last column.

Table 4.2 Parametric compilation, C engine versus `cvxcore`: first solve and warm re-solve. Ratio is on the re-solve.

| | | First solve (s) | | Re-solve (s) | | |
|---|---|---|---|---|---|---|
| Benchmark | #par | `CPP` | engine | `CPP` | engine | Ratio |
| ParamConeMatrixStuffing | 3 | 0.31 | 0.14 | 0.0030 | 0.0010 | **0.33** |
| ParamSmallMatrixStuffing | 3 | 0.48 | 0.21 | 0.0048 | 0.0017 | 0.35 |
| SimpleFullyParametrizedLP | 1 | 2.12 | 0.45 | 0.230 | 0.123 | 0.54 |
| ParametrizedQP | 2 | 10.19 | 2.25 | 1.361 | 0.959 | 0.70 |
| SimpleScalarParametrizedLP | 1 | 5.29 | 0.92 | 0.330 | 0.251 | 0.76 |
| ConvexPlasticity | 3 | 2.09 | 2.13 | 0.668 | 0.666 | 1.00 |
| FactorCovarianceModel | 2 | 5.24 | 0.73 | 0.144 | 0.277 | 1.92 |
| SVMWithL1Regularization | 1 | 11.16 | 1.60 | 0.263 | 0.530 | **2.01** |
| **Geometric mean** | | | | | | **0.78** |

The engine wins the re-solve on six of the eight parametric problems and the first solve on all but ConvexPlasticity. Because the engine's program is cached, a re-solve only re-evaluates the parameter-dependent coefficient entries; the DPP baseline instead multiplies its cached parameter tensor by the new parameter vector, which is hard to beat when that tensor is small and dense. The two rows where DPP retains a $\approx 2\times$ edge share one trait: a scalar parameter whose effect is spread across many coefficients (SVM's regularization weight, FactorCovariance's risk aversion multiplying the quadratic), the ideal case for the tensor product and the worst case for entry-wise re-evaluation.

### 4.4 Comparison against CasADi

CasADi [14] extracts problem data by algorithmic differentiation of a symbolic expression graph. Since CasADi has no notion of disciplined convex programming, it cannot canonicalize

the high-level models; for a fair comparison we hand it the *already-lowered* formulation: the same epigraph variables and the same conic rows that CVXPY's canonicalization produces, written as CasADi `MX` expressions $f(z, \theta)$ (objective) and $g(z, \theta) = b - Az$ (constraint slacks), where $z$ stacks the original and epigraph variables and $\theta$ collects the parameters. The problem data are recovered exactly the way CasADi's own quadratic-programming interface does, by differentiating the graph with no solver involved: $c = \nabla_z f|_{z=0}$, $P = \partial(\nabla_z f)/\partial z$, $A = -\partial g/\partial z$, and $b = g|_{z=0}$, wrapped in a `casadi.Function` mapping $\theta \mapsto (P, c, A, b)$; $P$ is computed as the Jacobian of the gradient, CasADi's fastest path on these problems. The *single* regime times differentiation, function construction, and the first numeric evaluation on a freshly built graph; the *re-solve* regime times one further evaluation at fresh parameter values on the compiled function. Graph construction itself is excluded, mirroring the engine's timing, which also starts from a built expression tree. We verified on every benchmark that the two systems produce identical $(P, c, A, b)$ at full benchmark size, so the tables below time the construction of the same matrices. All 25 problems are compared, with CVaR reported as a did-not-finish.

The single extraction of a freshly built problem is reported in Table 4.3, mirroring Table 4.1.

The engine is fastest where the lowered matrices contain large dense blocks: it knows every atom's Jacobian block and its sparsity pattern in closed form, while CasADi must discover the pattern and reconstruct the values through seeded derivative sweeps, whose number a dense block forces to the full dimension (on CVaR's dense scenario block the extraction did not finish within 10 h). The ratio reverses on the smallest problems, where the engine column is dominated not by extraction but by CVXPY's Python-side reduction chain, which the hand-lowered CasADi models are spared by construction.

On the parametric subset we report the first extraction and the warm re-solve (Table 4.4), mirroring Table 4.2.

The re-solve is CasADi's home regime: evaluating the compiled function tape beats the engine's re-solve, which still traverses CVXPY's solve path, by about 2× in geometric mean, and dramatically so when the parameters only touch $b$ (ConvexPlasticity). The engine retains the advantage when a parameter reshapes the quadratic itself (FactorCovarianceModel), where the tape must re-gather a large parameter-dependent $P$ on every evaluation.

Table 4.3 Single problem-data extraction across all 25 benchmarks, C engine versus CasADi (s).

| Benchmark | CasADi (s) | C engine (s) | Ratio |
|---|---|---|---|
| ParametrizedQP | 771.5 | 2.52 | **0.003** |
| FactorCovarianceModel | 250.4 | 2.05 | 0.008 |
| SVMWithL1Regularization | 145.5 | 1.79 | 0.012 |
| SemidefiniteProgramming | 66.4 | 2.07 | 0.031 |
| SimpleQP | 53.7 | 1.70 | 0.032 |
| LeastSquares | 36.6 | 1.62 | 0.044 |
| HuberRegression | 42.4 | 1.88 | 0.044 |
| Yitzhaki | 8.72 | 0.47 | 0.054 |
| SlowPruning | 38.8 | 2.80 | 0.072 |
| SimpleLP | 38.1 | 5.02 | 0.132 |
| TvInpainting | 8.49 | 1.14 | 0.134 |
| Murray | 25.9 | 3.52 | 0.136 |
| SimpleFullyParametrizedLP | 2.36 | 0.48 | 0.204 |
| SimpleScalarParametrizedLP | 4.57 | 0.98 | 0.214 |
| Cajas | 4.88 | 1.12 | 0.229 |
| ConeMatrixStuffing | 6.10 | 3.94 | 0.646 |
| OptimalAdvertising | 9.47 | 6.16 | 0.650 |
| SDPSegfault1132 | 17.65 | 14.61 | 0.83 |
| SmallMatrixStuffing | 2.77 | 2.75 | 0.99 |
| QuantumHilbertMatrix | 1.41 | 1.57 | 1.11 |
| UnconstrainedQP | 0.021 | 0.074 | 3.49 |
| ConvexPlasticity | 0.13 | 2.29 | 17.3 |
| ParamSmallMatrixStuffing | 0.007 | 0.21 | 30.8 |
| ParamConeMatrixStuffing | 0.004 | 0.14 | **39.7** |
| CVaR | DNF | 25.3 | — |
| **Geometric mean** | | | **0.24** |

Table 4.4 Parametric extraction, C engine versus CasADi: first extraction and warm re-solve (s). Ratio is on the re-solve.

| | | First (s) | | Re-solve (s) | | |
|---|---|---|---|---|---|---|
| Benchmark | #par | CasADi | engine | CasADi | engine | Ratio |
| FactorCovarianceModel | 2 | 250.4 | 2.05 | 3.956 | 0.567 | **0.14** |
| SVMWithL1Regularization | 1 | 145.5 | 1.79 | 0.548 | 0.491 | 0.90 |
| ParametrizedQP | 2 | 771.5 | 2.52 | 0.938 | 0.942 | 1.00 |
| SimpleScalarParametrizedLP | 1 | 4.57 | 0.98 | 0.124 | 0.236 | 1.90 |
| SimpleFullyParametrizedLP | 1 | 2.36 | 0.48 | 0.062 | 0.120 | 1.95 |
| ParamSmallMatrixStuffing | 3 | 0.007 | 0.21 | 0.0007 | 0.0020 | 3.0 |
| ParamConeMatrixStuffing | 3 | 0.004 | 0.14 | 0.0003 | 0.0011 | 3.5 |
| ConvexPlasticity | 3 | 0.13 | 2.29 | 0.0067 | 0.707 | **105.5** |
| **Geometric mean** | | | | | | **2.19** |

# CHAPTER 5 CONCLUSION

## 5.1 Summary of Works

## 5.2 Limitations

## 5.3 Future Research

## REFERENCES


[1] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[2] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[3] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[4] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[5] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[6] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

[7] A. R. Curtis, M. J. D. Powell, and J. K. Reid, "On the estimation of sparse Jacobian matrices," *IMA Journal of Applied Mathematics*, vol. 13, no. 1, pp. 117–119, 1974. [Online]. Available: https://doi.org/10.1093/imamat/13.1.117

[8] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? Graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005.

[9] C. F. Kwok, D. Zhu, and L. Jacobi, "An analysis of vectorised automatic differentiation for statistical applications," *Stats*, vol. 8, no. 2, 2025. [Online]. Available: https://doi.org/10.3390/stats8020040

[10] Y. Zou and F. Jiang, "Vectorized sparse second-order forward automatic differentiation for optimal control direct methods," *arXiv preprint arXiv:2506.11537*, 2025. [Online]. Available: https://arxiv.org/abs/2506.11537

[11] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[12] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[13] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[14] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[15] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[16] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[17] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[18] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[19] J. Bisschop, *AIMMS optimization modeling*, 2006.

[20] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide*. Redwood City, CA: The Scientific Press, 1988.

[21] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[22] M. L. Bynum *et al.*, *Pyomo—Optimization Modeling in Python*. Springer, 2021.

[23] W. E. Hart *et al.*, "Pyomo: modeling and solving mathematical programs in python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[24] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[25] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[26] A. Fu, B. Narasimhan, and S. Boyd, "Cvxr: An r package for disciplined convex optimization," *Journal of Statistical Software*, vol. 94, no. 14, 2020. [Online]. Available: http://dx.doi.org/10.18637/jss.v094.i14

[27] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[28] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[29] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[30] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[31] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," 2025.

[32] W. Zhang, P. Nobel, A. Jeendgar, R. Murray, P. Schiele, and S. Diamond, "CVXPY 1.9: Recent advances in optimization modeling software," https://www.cvxpy.org/_static/cvxpy-1.9.pdf, 2026, preprint.

[33] E. M. B. Smith, "On the optimal design of continuous processes," Ph.D. dissertation, Imperial College London (University of London), London, 1996, ph.D. dissertation.

[34] M. Grant and S. Boyd, "Graph Implementations for Nonsmooth Convex Programs," in *Recent Advances in Learning and Control*, ser. Lecture Notes in Control and Information Sciences, V. Blondel, S. Boyd, and H. Kimura, Eds. Springer, London, 2008, vol. 371. [Online]. Available: https://doi.org/10.1007/978-1-84800-155-8_7

[35] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2649-2

[36] E. Chu, N. Parikh, A. Domahidi, and S. Boyd, "Code generation for embedded second-order cone programming," in *2013 European Control Conference (ECC)*, 2013, pp. 1547–1552.

[37] M. ApS, "MOSEK modeling cookbook," 2026.

[38] J. R. Magnus and H. Neudecker, *Matrix Differential Calculus with Applications in Statistics and Econometrics*, 3rd ed., ser. Wiley Series in Probability and Statistics. John Wiley & Sons, 2019.

[39] P. S. Dwyer and M. S. Macphail, "Symbolic matrix derivatives," *The Annals of Mathematical Statistics*, vol. 19, no. 4, pp. 517–534, 1948.

[40] G. Guennebaud, B. Jacob *et al.*, "Eigen v3," 2010. [Online]. Available: http://eigen.tuxfamily.org

[41] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[42] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *J. Mach. Learn. Res.*, vol. 18, no. 1, p. 5595–5637, Jan. 2017.

[43] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2008.

[44] P. C. Chakravarti, "Truncation error in interpolation and numerical differentiation," *Numer. Math.*, vol. 3, no. 1, p. 279–284, Dec. 1961. [Online]. Available: https://doi.org/10.1007/BF01386029

[45] P. Wolfe, "Checking the calculation of gradients," *ACM Trans. Math. Softw.*, vol. 8, no. 4, p. 337–343, Dec. 1982. [Online]. Available: https://doi.org/10.1145/356012.356013

[46] Wolfram Research, Inc., "Mathematica, Version 14.3," 2026, champaign, IL. [Online]. Available: https://www.wolfram.com/mathematica

[47] Maplesoft, a division of Waterloo Maple Inc., "Maple," Waterloo, Ontario. [Online]. Available: https://www.maplesoft.com

[48] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, "Sympy: symbolic computing in python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. [Online]. Available: https://doi.org/10.7717/peerj-cs.103

[49] S. Laue, "On the equivalence of forward mode automatic differentiation and symbolic differentiation," *ArXiv*, vol. abs/1904.02990, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:102350450

[50] U. Naumann, *The Art of Differentiating Computer Programs: An Introduction to Algorithmic Differentiation*, ser. Software, Environments, and Tools. Philadelphia, PA: SIAM, 2011.

[51] A. Griewank, D. Juedes, and J. Utke, "Algorithm 755: ADOL-C: a package for the automatic differentiation of algorithms written in C/C++," *ACM Transactions on Mathematical Software*, vol. 22, no. 2, pp. 131–167, 1996. [Online]. Available: https://doi.org/10.1145/229473.229474

[52] B. M. Bell, "CppAD: A package for C++ algorithmic differentiation," 2023, computational Infrastructure for Operations Research. [Online]. Available: https://github.com/coin-or/CppAD

[53] U. Naumann, "Optimal Jacobian accumulation is NP-complete," *Mathematical Programming*, vol. 112, no. 2, pp. 427–441, 2008. [Online]. Available: https://doi.org/10.1007/s10107-006-0042-z

[54] T. F. Coleman and J. J. Moré, "Estimation of sparse Jacobian matrices and graph coloring problems," *SIAM Journal on Numerical Analysis*, vol. 20, no. 1, pp. 187–209, 1983. [Online]. Available: https://doi.org/10.1137/0720013

[55] A. H. Gebremedhin, A. Tarafdar, A. Pothen, and A. Walther, "Efficient computation of sparse hessians using coloring and automatic differentiation," *INFORMS J.*

*on Computing*, vol. 21, no. 2, p. 209–223, Apr. 2009. [Online]. Available: https://doi.org/10.1287/ijoc.1080.0286

[56] A. Hill and G. Dalle, "Sparser, better, faster, stronger: Sparsity detection for efficient automatic differentiation," *arXiv preprint arXiv:2501.17737*, 2025. [Online]. Available: https://arxiv.org/abs/2501.17737

[57] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[58] L. Hascoët and V. Pascual, "The Tapenade automatic differentiation tool: Principles, model, and specification," *ACM Transactions on Mathematical Software*, vol. 39, no. 3, 2013. [Online]. Available: https://doi.org/10.1145/2450153.2450158

[59] W. Moses and V. Churavy, "Instead of rewriting foreign code for machine learning, automatically synthesize fast gradients," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 12 472–12 485.

[60] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: an imperative style, high-performance deep learning library.* Red Hook, NY, USA: Curran Associates Inc., 2019.

[61] M. Chi and S. Wen, "torch-sla: Differentiable sparse linear algebra with adjoint solvers and sparse tensor parallelism for PyTorch," *arXiv preprint arXiv:2601.13994*, 2026. [Online]. Available: https://arxiv.org/abs/2601.13994

[62] J. Hückelheim, M. Schanen, S. H. K. Narayanan, and P. Hovland, "Vector forward mode automatic differentiation on SIMD/SIMT architectures," in *Proceedings of the 49th International Conference on Parallel Processing (ICPP).* Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3404397.3404470

[63] A. Shaikhha, M. Huot, and S. Hashemian, "∇sd: Differentiable programming for sparse tensors," *arXiv preprint arXiv:2303.07030*, 2023. [Online]. Available: https://arxiv.org/abs/2303.07030

[64] A. Hill and G. Dalle, "An illustrated guide to automatic sparse differentiation," ICLR 2025 Blogposts Track, 2025. [Online]. Available: https://iclr-blogposts.github.io/2025/blog/sparse-autodiff/

[65] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[66] D. M. Gay, "Hooking your solver to AMPL," Bell Laboratories, Tech. Rep. 97-4-06, 1997, reference for the AMPL Solver Library (ASL).

[67] ——, "Writing .nl files," Sandia National Laboratories, Tech. Rep. SAND2005-7907P, 2005, describes AMPL's expression DAG and coefficient extraction internals.

[68] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[69] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2003.

[70] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2006. [Online]. Available: https://epubs.siam.org/doi/book/10.1137/1.9780898718881

[71] F. G. Gustavson, "Two fast algorithms for sparse matrices: Multiplication and permuted transposition," *ACM Transactions on Mathematical Software*, vol. 4, no. 3, pp. 250–269, 1978.

[72] R. E. Bank and C. C. Douglas, "Sparse matrix multiplication package (SMMP)," *Advances in Computational Mathematics*, vol. 1, no. 1, pp. 127–137, 1993.

[73] I. S. Duff, A. M. Erisman, and J. K. Reid, *Direct Methods for Sparse Matrices*. Oxford, England: Oxford University Press, 1986.

[74] P. E. Gill, W. Murray, M. A. Saunders, and M. H. Wright, "Sparse matrix methods in optimization," *SIAM Journal on Scientific and Statistical Computing*, vol. 5, no. 3, pp. 562–589, 1984.

[75] CVXPY developers, "CVXPY benchmarks," https://github.com/cvxpy/benchmarks, 2025.

[76] airspeed velocity developers, "airspeed velocity," https://asv.readthedocs.io/, 2025, python benchmarking framework.

[77] P. J. Goulart and Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives," *arXiv preprint arXiv:2405.12762*, 2024.